SOFTWARE ENGINEERING

Chapter 4

Implementation/Programming

Outline

- Design and implementation
- Types of Programming Methodologies
- Programming Language
- Programming style
- Implementation issues
- Open-source development

Design and implementation

- Software design and implementation is the stage in the software engineering process at which an executable software system is developed.
- Software design and implementation activities are invariably inter-leaved.
  - Software design is a creative activity in which you identify software components and their relationships, based on a customer's requirements.
  - Implementation is the process of realizing the design as a program.

Build or buy

- In a wide range of domains, it is now possible to buy off-the-shelf systems (COTS) that can be adapted and tailored to the users' requirements.
  - For example, if you want to implement a medical records system, you can buy a package that is already used in hospitals. It can be cheaper and faster to use this approach rather than developing a system in a conventional programming language.
- When you develop an application in this way, the design process becomes concerned with how to use the configuration features of that system to deliver the system requirements.

Goals of Implementation

- Implementation is done by the coder or programmers who are independent people than the designer.
- Goals:
  - *To translate the design of system into a computer language format*
  - *To reduce the cost of later phases*: The cost of testing and maintenance can be significantly reduced with efficient coding.
  - *Making the program more readable* → producing more maintainable software

Types of Programming Methodologies

- **Procedural Programming**
  - Problem is broken down into procedures
  - **Example:** For a calculator program that does addition, subtraction, multiplication, division, square root and comparison, each of these operations can be developed as separate procedures.
- **Object-oriented Programming**
  - The solution revolves around entities or objects that are part of problem.
  - **Example:** To develop a payroll management system, we will have entities like employees, salary structure, leave rules, etc. around which the solution must be built.

Types of Programming Methodologies

- **Functional Programming**
  - the problem, or the desired solution, is broken down into functional units. Each unit performs its own task
  - **Example**: A payroll processing can have functional units like employee data maintenance, basic salary calculation, gross salary calculation, leave processing, loan repayment processing, etc.
- **Logical Programming**
  - The Computer Programming Paradigm, in which the program statements express the facts and rules about different problems within a system of formal logic.
  - The rules are written in the form of logical clauses
    - canfly(X) :- bird(X), not abnormal(X).

Programming Language (PL)

- Characteristics of programming language
- Application domain of programming language
- Choice of programming language

Characteristics of PL

- **Capabilities**
  - Simple, concise, and intuitive syntax
  - Support multiple data types
  - Support pointer, recursion
  - Support object oriented programming
  - Rich library
- **Portability**
  - Programs can be compiled and run on multiple machines without the source program have to be re-written

Characteristics of PL

- **Support of tools**
  - Efficient compiler
    - High speed translation
    - High optimization ability
  - Other tools:
    - Editor, debugger, linker, …
    - IDE (Integrated Development Environment)

Application domain of PL

- Systems programming applications
  - fast execution, low-level featurs to access external devices
  - popular language: C (almost the whole of UNIX is written in C)
- Real-time systems: C, C++, Ada, Assembler
- Embedded software applications: C++, Java
- Scientific applications:
  - typically require simple data structures but a large number of flating-point operations
  - Fortran,Matlab,…
- Artificial intelligence applications
  - LISP, PROLOG, PYTHON,…

Application domain of PL

- Web Programming/CGI:
  - Perl, ASP, PHP, Java, JavaScript, Python..
- Mobile Programming:
  - Java/Kotlin (Android), Object-C/Swift (iOS), Javascript/React Native, Dart/Flutter (Cross Platforms)

Choice of programming language

- The factors we consider when determining the programming language for each project:
  - Characteristics of programming language
  - Application domain of programming language
  - Work environment
  - Speed of development or speed of execution
  - Expertise of the programmer
  - Complexity of the application
  - …

Programming style

- Programming style is set of coding rules followed by all the programmers to write the code.
- Good programming style: to make programs more comfortable to read and maintain

1. **Naming conventions**
   - global variable names always begin with a capital letter
   - local variable names are made of small letters
   - constant names are always capital letters
2. **Comment**
   - Inline comments promote readability
3. **Indent**
   - This is the space left at the beginning of line, usually 2-8 whitespace or single tab.

Programming style

Good programming style

4. **Structured Programming**

- "GOTO" statements shall not be used as they lead to "spaghetti" code, which is hard to read and maintain

5. Line Length: length of source code lines at or below 80 characters.

6. Error handling

- A mechanism to handle runtime errors
- The core advantage of exception handling is to maintain the normal flow of the application
- Separation of Error Handling code from Normal Code

Programming style

Throw the exception

```
double MyDivide(double num, double denom)  {
   if (denom == 0.0) {
      throw invalid_argument(”The denom cannot be 0.”);
      }
   else {
      return num / denom;
      }
}
```

Programming style

Catch the exception

```
try {
    result = MyDivide(x, y);
  }
  catch (invalid_argument& e) {
    cerr << e.what() << endl;
    ... // mã xử lý với ngoại lệ
  };
```

Efficient Programming

- **Efficiency of the algorithm** is determined in the detailed design. The two factors of Algorithm Complexity
  - **Time**: the amount of time that is required by the algorithm to execute and get the result (operations, conditional if-else statements, loop statements)
  - **Space**: the amount of memory required by the algorithm to store the variables and get the result (inputs, temporary operations, or outputs)
- Programming style can impact execution speed and memory requirements.

Efficient Programming

- **Guidelines**
  - Simplify arithmetic and logical expressions.
  - Carefully calculate each nested cycle to determine whether statements or expressions can be moved outside
  - Avoid using multi-dimensional arrays, pointers, recursion,…

Efficient Programming

Example 1: Find 1!,2!,3!...n!

- Solution 1:

  Step 1: Write a function float factorial(int k)

  Step 2:

  ```
  for i=1,...,n
      print factorial(i)
  ```

  Time Complexity: $O(n^2)$

Efficient Programming

- Solution 2:

  Using formula k!=(k-1)!*k.

  GT = 1

  for i=1,…,n

  GT=GT*i

  print GT

  Time Complexity: O(n)

Efficient Programming

Example 2: Find C^0_n, C^1_n,.., C^n_n

- Solution 1:

  - Write a function to find k!

  - Calculate: $C^k_n = n!/(k!*(n-k)!)$

  Time Complexity $O(n^2)$

Efficient Programming

- Solution 2
  - Using array with n+1 elements

    a[0],a[1],..,a[n]

    a[i]=i!

    Using loop to calculate C^k_n

    Time Complexity: $O(\alpha n)$

Efficient Programming

- Solution 3

$C^k_n = n!/(k!*(n-k)!)$

$C^{k+1}_n = n!/((k+1)!*(n-k-1)!) = n!/(k!*(k+1)*(n-k-1)!)$

$\Rightarrow C^{k+1}_n / C^k_n = (n-k)/(k+1) \Rightarrow C^{k+1}_n = ((n-k)/(k+1))*C^k_n$

$C^0_n = 1$

Time Complexity: O(n)

Not use memory to store the array

Implementation issues

- Focus here is not on programming, although this is obviously important, but on other implementation issues that are often not covered in programming texts:
  - Reuse: Most modern software is constructed by reusing existing components or systems. When you are developing software, you should make as much use as possible of existing code.
  - Configuration management: During the development process, you have to keep track of the many different versions of each software component in a configuration management system.
  - Host-target development: Production software does not usually execute on the same computer as the software development environment. Rather, you develop it on one computer (the host system) and execute it on a separate computer (the target system).

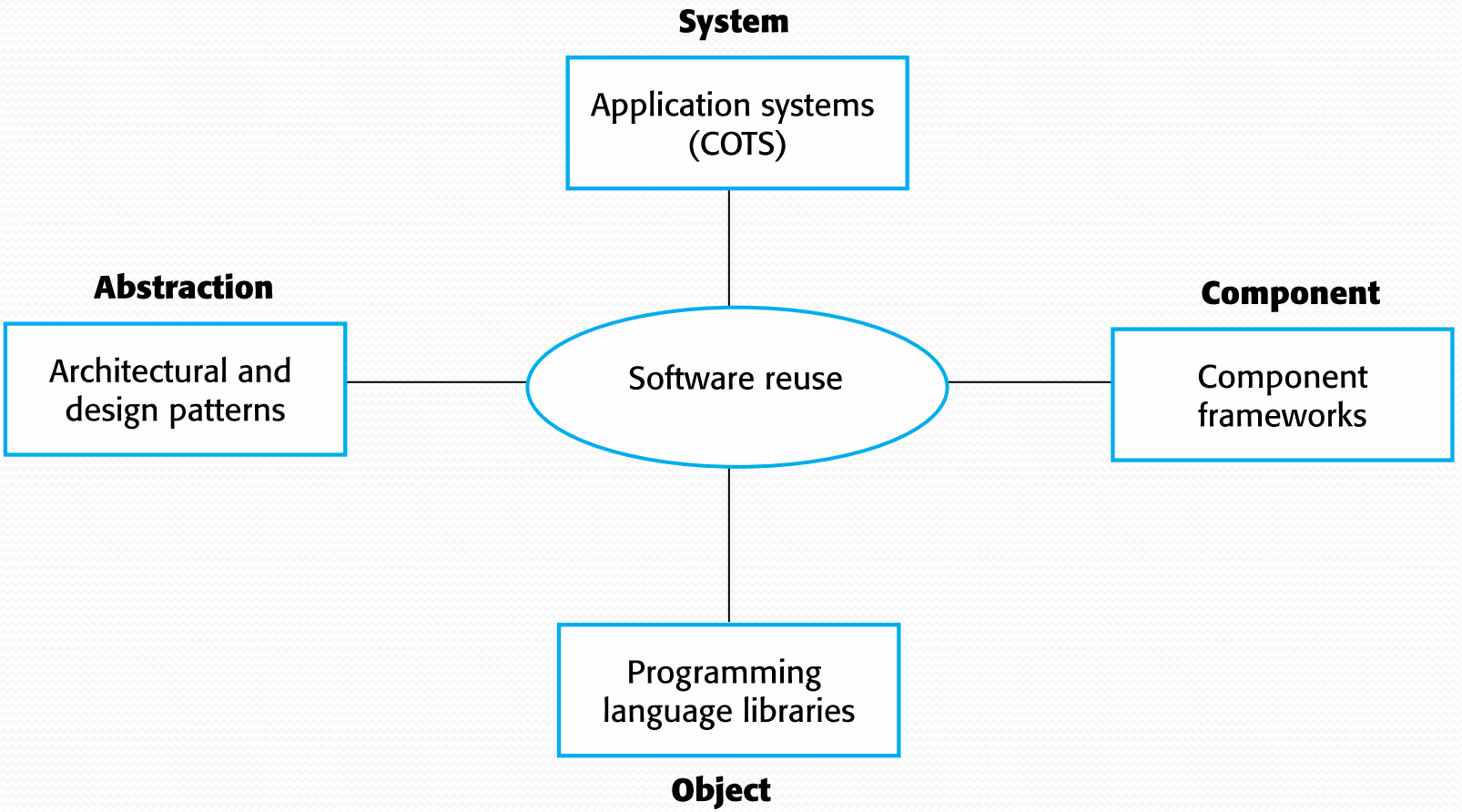
Reuse

- From the 1960s to the 1990s, most new software was developed from scratch, by writing all code in a high-level programming language.
  - The only significant reuse or software was the reuse of functions and objects in programming language libraries.
- Costs and schedule pressure mean that this approach became increasingly unviable, especially for commercial and Internet-based systems.
- An approach to development based around the reuse of existing software emerged and is now generally used for business and scientific software.

Reuse levels

- The abstraction level
  - At this level, you don’t reuse software directly but use knowledge of successful abstractions in the design of your software.
- The object level
  - At this level, you directly reuse objects from a library rather than writing the code yourself.
- The component level
  - Components are collections of objects and object classes that you reuse in application systems. (framework)
- The system level
  - At this level, you reuse entire application systems.

Software reuse



Reuse costs

- The costs of the time spent in looking for software to reuse and assessing whether or not it meets your needs.
- Where applicable, the costs of buying the reusable software. For large off-the-shelf systems, these costs can be very high.
- The costs of adapting and configuring the reusable software components or systems to reflect the requirements of the system that you are developing.
- The costs of integrating reusable software elements with each other (if you are using software from different sources) and with the new code that you have developed.

Configuration management

- Configuration management is the name given to the general process of managing a changing software system.
- The aim of configuration management is to support the system integration process so that all developers can access the project code and documents in a controlled way, find out what changes have been made, and compile and link components to create a system.

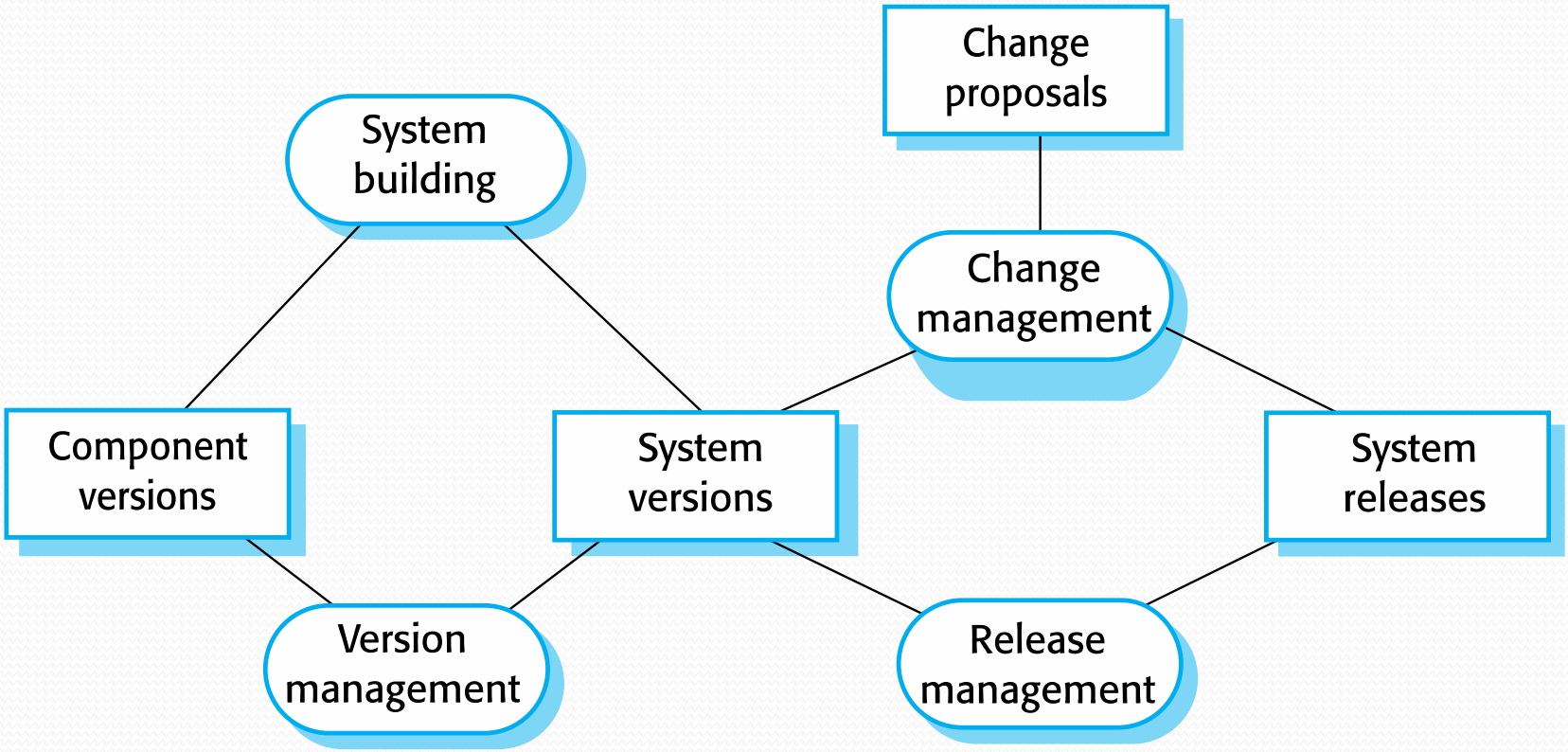
Configuration management tools



Configuration management activities

- Version management: keep track of the different versions of software components. Version management systems include facilities to coordinate development by several programmers.
- System integration: help developers define what versions of components are used to create each version of a system. This description is then used to build a system automatically by compiling and linking the required components.
- Problem tracking: allow users to report bugs and other problems, and to allow all developers to see who is working on these problems and when they are fixed.

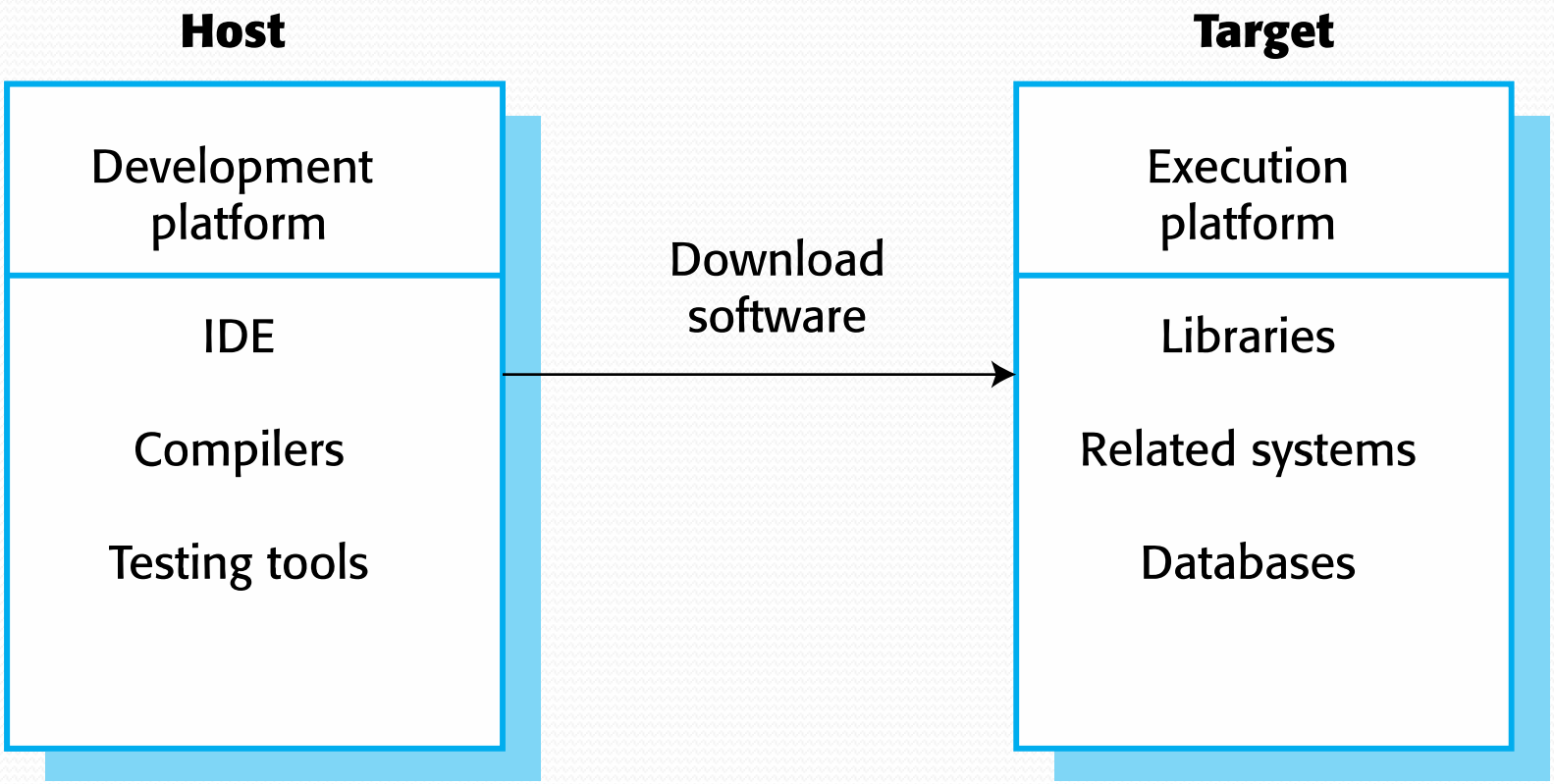
Configuration management tool interaction



Host-target development

- Most software is developed on one computer (the host), but runs on a separate machine (the target).
- More generally, we can talk about a development platform and an execution platform.
  - A platform is more than just hardware.
  - It includes the installed operating system plus other supporting software such as a database management system or, for development platforms, an interactive development environment.
- Development platform usually has different installed software than execution platform; these platforms may have different architectures.

Host-target development



Development platform tools

- An integrated compiler and syntax-directed editing system that allows you to create, edit and compile code.
- A language debugging system.
- Graphical editing tools, such as tools to edit UML models.
- Testing tools, such as Junit that can automatically run a set of tests on a new version of a program.
- Project support tools that help you organize the code for different development projects.

Integrated development environments (IDEs)

- Software development tools are often grouped to create an integrated development environment (IDE).
- An IDE is a set of software tools that supports different aspects of software development, within some common framework and user interface.
- IDEs are created to support development in a specific programming language such as Java. The language IDE may be developed specially, or may be an instantiation of a general-purpose IDE, with specific language-support tools.

Open source development

Open source development

- Open source development is an approach to software development in which the source code of a software system is published and volunteers are invited to participate in the development process
- Its roots are in the Free Software Foundation (www.fsf.org), which advocates that source code should not be proprietary but rather should always be available for users to examine and modify as they wish.
- Open source software extended this idea by using the Internet to recruit a much larger population of volunteer developers. Many of them are also users of the code.

Open source systems

- The most famous open source product is the Linux operating system, which is widely used as a server system (CentOS, ...) and also for desktop computers (Ubuntu, ...)
- Other important open source products are Apache (Web Server) and MySQL (MariaDB), and many open source applications are currently in common use such as: WordPress (CMS), Magento (E-commerce), ...

Open source issues

- Should the product that is being developed make use of open source components?
- Should an open source approach be used for the software's development?

Open source business

- More and more product companies are using an open source approach to development.
- Their business model is not reliant on selling a software product but on selling support for that product.
- They believe that involving the open source community will allow software to be developed more cheaply, more quickly and will create a community of users for the software.

Open source licensing

- A fundamental principle of open-source development is that source code should be freely available, this does not mean that anyone can do as they wish with that code.
  - Legally, the developer of the code (either a company or an individual) still owns the code. They can place restrictions on how it is used by including legally binding conditions in an open source software license.
  - Some open source developers believe that if an open source component is used to develop a new system, then that system should also be open source.
  - Others are willing to allow their code to be used without this restriction. The developed systems may be proprietary and sold as closed source systems.

License models

- **The GNU General Public License (GPL)**. This is a so-called ‘reciprocal’ license that means that if you use open source software that is licensed under the GPL license, then you must make that software open source.
- **The GNU Lesser General Public License (LGPL)** is a variant of the GPL license where you can write components that link to open source code without having to publish the source of these components.
- **The Berkley Standard Distribution (BSD) License**. This is a non-reciprocal license, which means you are not obliged to re-publish any changes or modifications made to open source code. You can include the code in proprietary systems that are sold.

License management

- Establish a system for maintaining information about open-source components that are downloaded and used.
- Be aware of the different types of licenses and understand how a component is licensed before it is used.
- Be aware of evolution pathways for components.
- Educate people about open source.
- Have auditing systems in place.
- Participate in the open source community.

Key points

- Software design and implementation are inter-leaved activities. The level of detail in the design depends on the type of system and whether you are using a plan-driven or agile approach.
- When developing software, you should always consider the possibility of reusing existing software, either as components, services or complete systems.

Key points

- Configuration management is the process of managing changes to an evolving software system. It is essential when a team of people are cooperating to develop software.
- Most software development is host-target development. You use an IDE on a host machine to develop the software, which is transferred to a target machine for execution.
- Open source development involves making the source code of a system publicly available. This means that many people can propose changes and improvements to the software.